

Shakespeare-based Language Modeling: from Classical N-Gram Models to GPT-2

Ali Dulaimi

August - 2025

GPT from Scratch by Elia Bruni

Project Overview

This project takes a deep dive into language modeling, starting with the basics of tokenization and moving all the way to modern transformer architectures. Using Shakespeare's complete works as our dataset, we built and compared four different approaches to language modeling and text generation. The result is a full pipeline that shows how natural language processing techniques have evolved over time.

The project was structured as four progressive tasks:

- **Task 1:** Byte-Pair Encoding (BPE) tokenization with optimization
- **Task 2:** Classical n-gram language models with statistical smoothing
- **Task 3:** Neural n-gram models with embedding layers
- **Task 4:** GPT-style transformer architecture

Each task builds upon the previous one, allowing us to compare different approaches on the same dataset and understand the trade-offs between classical statistical methods and modern neural techniques.

How to Run the Code

Setup and Dependencies

You'll need a Python environment with PyTorch, matplotlib, numpy, and a few other ML libraries installed. The easiest way is to use [Google Colab with a GPU enabled](#). When you start running the code, it will automatically download the Shakespeare dataset from Google Drive. From there, it runs through four main stages in order:

- BPE tokenization evaluation
- Classical n-gram models
- Neural n-gram models
- GPT-style transformer training

Execution

Just run the file from top to bottom, it's set up to work as a single pipeline. Built-in caching makes sure you don't redo expensive computations unnecessarily. Each stage saves its results in its own directory, and later stages build on the earlier ones.

Be aware: the full pipeline takes several hours, mainly because of neural network training. While it runs, you'll see progress updates and memory usage logs. Once finished, all results, both JSON files and plots are automatically saved.

Code can be accessed through [this link](#). For a quick demo on HuggingFace that includes the best performing models from our notebook can be found [here](#). Finally, the github repo with all files, demo, etc [here](#).

Task 1: Byte-Pair Encoding (BPE) Tokenization

Theoretical Background

Byte-Pair Encoding (BPE) tackles a key problem in natural language processing: deciding how to break text into useful pieces. Older methods worked at the character level (too small and detailed) or the word level (too rigid and unable to handle new words). BPE finds a middle ground by learning subword units—pieces of words that are flexible like characters but still carry meaning like words.

The algorithm works iteratively:

- Start with all characters as individual tokens
- Find the most frequent pair of adjacent tokens
- Merge this pair into a single new token
- Repeat for a predetermined number of merges

For example, with text "the cat and the dog":

- Initial: ["t", "h", "e", " ", "c", "a", "t", " ", " ", "a", "n", "d", " ", "t", "h", "e", " ", "d", "o", "g"]
- After merging "t,h": ["th", "e", " ", "c", "a", "t", " ", "a", "n", "d", " ", "th", "e", " ", "d", "o", "g"]
- After merging "th,e": ["the", " ", "c", "a", "t", " ", "a", "n", "d", " ", "the", " ", "d", "o", "g"]

Implementation Details

Our BPE implementation includes several key components:

```
class BPE:
    def _stats(self, tokens):
        """Count adjacent token pairs"""
        pairs = Counter()
        for i in range(len(tokens) - 1):
            pairs[(tokens[i], tokens[i+1])] += 1
        return pairs

    def _merge_vocab(self, pair, tokens):
        """Merge all instances of pair in token sequence"""
        a, b = pair
        ab = a + b
        new_tokens = []
        i = 0
        while i < len(tokens):
            if i < len(tokens) - 1 and tokens[i] == a and tokens[i+1] == b:
                new_tokens.append(ab)
                i += 2
            else:
                new_tokens.append(tokens[i])
                i += 1
        return new_tokens
```

The evaluation focused on two key metrics:

- **Tokens per word (TPW):** Lower values indicate better compression
- **Vocabulary efficiency:** Balance between vocabulary size and compression quality

We tested two normalization strategies:

- **minimal_clean:** Basic lowercase and space cleanup
- **lower_nopunct:** Aggressive punctuation removal and lowercasing

Task 1 Results

Testing across 5 merge counts (500, 1000, 1500, 2000, 2500) and 2 normalization strategies revealed clear patterns:

Key findings:

500	minimal_clean	536	2.177	TRUE	1166.9
1000	minimal_clean	1031	1.874	TRUE	1932.1
1500	minimal_clean	1523	1.695	TRUE	2581.5
2000	minimal_clean	2018	1.606	TRUE	3240.9
2500	minimal_clean	2510	1.523	TRUE	3822.7
500	lower_nopunct	523	1.771	TRUE	926.6
1000	lower_nopunct	1004	1.502	TRUE	1508
1500	lower_nopunct	1484	1.352	TRUE	2006.8
2000	lower_nopunct	1969	1.253	TRUE	2467.8
2500	lower_nopunct	2459	1.193	TRUE	2933.5

- **lower_nopunct** normalization dramatically outperformed **minimal_clean** by enabling better character pair merging across word boundaries
- More merges consistently reduced tokens per word but increased vocabulary size
- The 2500-merge configuration with **lower_nopunct** achieved the best compression (1.19 tokens/word) and was selected for subsequent tasks
- All configurations achieved perfect reconstruction accuracy

Task 2: Classical N-gram Language Models

Theoretical Background

N-gram models represent the classical approach to language modeling, based on the Markov assumption that the probability of a word depends only on the previous n-1 words. These models estimate probabilities by counting occurrences in training data:

$$P(w_i | w_{\{i-n+1\}}, \dots, w_{\{i-1\}}) = \text{Count}(w_{\{i-n+1\}}, \dots, w_i) / \text{Count}(w_{\{i-n+1\}}, \dots, w_{\{i-1\}})$$

However, data sparsity creates challenges - many possible n-grams never appear in training data. We addressed this through three techniques:

- **Add-one (Laplace) smoothing:** Add 1 to all counts to handle unseen n-grams (used to ensure safe probability calculations in all three cases)

- **Interpolation:** Blend probabilities from different n-gram orders
- **Backoff:** Use shorter n-grams when longer ones are unavailable

Implementation Details

Our n-gram implementation builds on the optimized BPE tokenizer from Task 1:

```
class NGramModel:
    def train_ngram(self, train_tokens, n, k=1.0):
        ng, ctx = Counter(), Counter()

        # Sentence-aware training to avoid cross-sentence n-grams
        for ln in self._train_lines:
            padded = [self.START]*(n-1) + ln + [self.END]
            for i in range(len(padded)-n+1):
                g = tuple(padded[i:i+n])
                ng[g] += 1
                if n > 1:
                    ctx[g[:-1]] += 1
```

The model supports three probability estimation methods:

- **Linear interpolation:** Weighted combination of different n-gram orders
- **Simple backoff:** Use highest-order n-gram with non-zero count

Task 2 Results

Using the optimal BPE configuration (2500 merges, lower_nopunct), we trained models from unigram to 4-gram:

Key insights:

Unigram	1221.1	1221.1	1221.1
Bigram	1051.26	915.96	904.13
Trigram	1974.73	1153.13	1008.88
4-gram	2218.8	1375.93	1025.52

- Bigram models achieved the best performance (904 perplexity) with simple backoff
- Add-k smoothing degraded performance for higher-order n-grams due to excessive

- smoothing over large BPE vocabularies
- Backoff consistently outperformed interpolation by avoiding over-smoothing
- The k-value sweep showed optimal performance around $k=0.01$, but we maintained $k=1.0$ per assignment requirements

Text generation samples revealed characteristic n-gram behavior:

- **Argmax:** Grammatical but repetitive outputs
- **Sampling:** More varied but occasionally incoherent text
- **Context sensitivity:** Clear improvements from unigram to trigram models

Task 3: Neural N-gram Language Models

Theoretical Background

Neural language models address the sparsity problem by learning dense vector representations (embeddings) that capture semantic similarity. Instead of discrete counts, these models use continuous representations where similar words have similar embeddings, allowing generalization beyond exact matches.

The architecture consists of:

- **Embedding layer:** Maps discrete tokens to dense vectors
- **Feedforward networks:** Process concatenated context embeddings
- **Output layer:** Predicts probability distribution over vocabulary

For a trigram model: $[\text{token}_1, \text{token}_2] \rightarrow [\text{emb}_1, \text{emb}_2] \rightarrow \text{concat} \rightarrow \text{MLPs} \rightarrow \text{softmax}$

Implementation Details

Our neural n-gram implementation uses PyTorch with careful regularization to prevent overfitting:

```
class NeuralNgramModel(nn.Module):
    def __init__(self, vocab_size, n, n_embd=256, n_hidden=512, dropout=0.2):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, n_embd)

        if n == 1:
            self.out = nn.Linear(n_embd, vocab_size)
        else:
            inp = n_embd * (n-1)
```

```
self.fc1 = nn.Linear(inp, n_hidden)
self.fc2 = nn.Linear(n_hidden, n_hidden//2)
self.out = nn.Linear(n_hidden//2, vocab_size)
```

Key training techniques included:

- **Early stopping:** Prevent overfitting with patience-based termination
- **Learning rate scheduling:** Adaptive reduction on validation plateau
- **Log-linear interpolation:** Blend neural predictions with unigram priors
- **Gradient clipping:** Stabilize training dynamics

Task 3 Results

Training neural models with grid search over optimizers (Adam, AdamW) and interpolation weights:

1-gram	1221.78	1208.6	632,734	45.9s
2-gram	315.21	291.52	1,525,918	173.5s
3-gram	253.47	220.56	1,656,990	261.5s
4-gram	260.16	220.11	1,788,062	244.0s

Major improvements over classical models:

- Neural 3-gram achieved 253 perplexity vs. 1009 for classical trigram
- Embeddings successfully captured semantic similarities for better generalization
- Adam optimizer with $\alpha=0.2$ interpolation weight proved optimal for $n \geq 2$

The 3-gram model found the sweet spot between context length and overfitting, while 4-gram showed slight degradation despite lower validation perplexity.

Task 4: GPT(2?) Language Modeling on Shakespeare

Theoretical Background

Transformers revolutionized NLP by replacing recurrent architectures with attention mechanisms that can process sequences in parallel. The key innovation is self-attention, which allows each

position to attend to all previous positions simultaneously.

The GPT architecture consists of:

- **Token + positional embeddings:** Represent both content and position
- **Causal self-attention:** Each token attends only to previous tokens
- **Layer normalization:** Stabilize training of deep networks
- **Feedforward blocks:** Process attended representations

Image source: <https://magazine.sebastianraschka.com/p/from-gpt-2-to-gpt-oss-analyzing-the>

The causal mask ensures autoregressive generation: $\text{mask}[i,j] = -\infty$ if $j > i$ else 0

Implementation Details

Our transformer implementation builds causal self-attention from scratch:

```
class CausalSelfAttention(nn.Module):
    def forward(self, x):
        B, T, C = x.shape
        q, k, v = self.c_attn(x).split(C, dim=2)

        # Reshape for multi-head attention
        q = q.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
        k = k.view(B, T, self.n_head, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.n_head, self.head_dim).transpose(1, 2)

        # Scaled dot-product attention with causal mask
        att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(self.head_dim))
        att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float("-inf"))
        att = F.softmax(att, dim=-1)

        y = att @ v
        return y.transpose(1, 2).contiguous().view(B, T, C)
```

To prevent overfitting on the limited Shakespeare dataset, we implemented aggressive regularization:

- High dropout rates (0.4-0.6)
- Low learning rates (8e-5)
- Label smoothing (0.1)
- Early stopping with patience=4
- Weight tying between embeddings and output layer

Task 4 Results

Training three model sizes with anti-overfitting configurations:

GPT-tiny	540.52	530.91	260,736	E64-H2-L2-B48
GPT-small	422.06	406.25	578,208	E96-H4-L3-B64
GPT-medium	392.14	352.59	1,120,768	E128-H4-L4-B96

Observations:

- GPT models didn't surpass classical n-grams in perplexity, highlighting the importance of data scale for transformers
- Larger models showed consistent improvements, suggesting the architecture could benefit from more parameters and data
- Generated text exhibited local coherence with Shakespearean vocabulary and style
- Training was stable across all configurations with consistent GPU memory usage

Sample generations showed promising quality:

"to be or not to be trace by our duty to your wife and brother s witness as i do
receive up with your mind i dare your heart continent and will never trust..."

Comparative Analysis and Lessons Learned

Performance Comparison

Classical N-gram	904	Simple, interpretable, efficient	Sparsity issues, no semantic understanding

Neural N-gram	253	Semantic embeddings, better generalization	Requires more data, computationally intensive
GPT Transformer	392	Parallel processing, flexible architecture	Needs massive scale to excel

Key Insights

- **Data Scale Matters:** Classical methods excel with limited data, while neural approaches need scale to demonstrate advantages
- **Architecture Complexity:** More sophisticated models require proportionally more data and computational resources
- **Tokenization Impact:** Proper BPE configuration (lower_nopunct normalization) provided crucial foundation for all subsequent models
- **Regularization Critical:** Neural models required extensive regularization to prevent overfitting on Shakespeare corpus
- **Evaluation Nuance:** Perplexity doesn't fully capture generation quality differences between architectures

Implementation Value

Building these models from scratch provided deep insights into:

- How tokenization affects downstream model performance
- Trade-offs between statistical and neural approaches
- Importance of proper regularization and training procedures
- The evolution from discrete counting to continuous representations

How to improve?

To bridge the performance gap and improve upon this work:

Immediate Improvements

- **Scale up training data:** Expand beyond Shakespeare to larger, more diverse corpora
- **Increase model size:** Test configurations with 10M+ parameters
- **Advanced architectures:** Implement layer normalization improvements, better attention

mechanisms

Research Directions

- **Hybrid approaches:** Combine n-gram insights with transformer architectures
- **Efficient training:** Investigate techniques for better performance on limited data
- **Evaluation metrics:** Develop measures beyond perplexity that capture generation quality

Technical Enhancements

- **Optimization:** Better learning rate schedules, optimizer configurations
- **Architecture search:** Automated discovery of optimal model configurations
- **Transfer learning:** Leverage pre-trained components where possible

In the final part of the notebook, I trained these models but with tiktoken (installed from github) tokenization algorithm to see the performance.

Results:

Superior performance on all three model types: classic n-gram modeling, neural n-gram modeling and GPT2-modeling. The generated texts were very high-quality and Shakespeare-like. The problem is, the model was suffering from severe overfitting to the degree of providing a perplexity value of around 10 on the training set and much higher perplexity on the test set. However the generations look perfectly Shakespeare-like. Tiktoken performed better mainly because of its much larger, professionally trained vocabulary (~50K tokens compared to our ~2.5K) and its smarter handling of subword boundaries, learned from a wide variety of text. This makes tokenization more efficient, fewer tokens are needed per word, so the model can cover longer spans of text within the same context window.

The downside was that this efficiency led to severe overfitting, since the model could memorize training patterns more easily. Still, the generations stayed strong because Tiktoken's tokens line up well with how language is structured. That alignment makes local text more coherent, even if the model struggles at a global scale when overfit.

Conclusion

This project was all about tracing how language modeling has grown, from the old-school statistical methods to the neural networks we use today. Our GPT model didn't beat the classic n-gram approach in perplexity, but that wasn't really the point, it gave us a solid way to understand how transformers work and how they can be scaled up into something more powerful.

We built the whole pipeline ourselves, starting with tokenization and ending with a working transformer. Along the way, we saw how the "best" method really depends on the data you have.

Simpler, classical approaches can still be surprisingly strong in small, well-defined settings, while neural networks start to shine once you scale up.

The biggest takeaway, though, was what we learned by implementing everything from scratch. It gave us a real feel for the decisions that shape modern language models—from the smallest choices about tokenization to the architectural tricks that make today's large models possible.

In the end, the project did what we set out to do: build a hands-on understanding of language modeling, while seeing both the strengths and limits of different approaches in practice.